Name: Mehak Fatima
Seat Number: B21110006057
Class: BSCS (4th Year)

ASSIGNMENT # 02

Generate 500 unique random numbers than apply any two searching and sorting algorithms on them and compare for efficiency.

SORTING

Code:

```
import random
import time

def bubble_sort(arr):
    comparisons = 0
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr, comparisons

def selection_sort(arr):
    comparisons = 0
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            comparisons += 1
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr, comparisons

# Generate 500 unique random numbers
data = random.sample(range(1, 10000), 500)

# Bubble Sort
bubble_data = data.copy()
start = time.perf_counter()
bubble_sorted, bubble_comps = bubble_sort(bubble_data)
bubble_time = time.perf_counter() - start

# Selection Sort
selection_data = data.copy()
start = time.perf_counter()
selection_sorted, selection_comps = selection_sort(selection_data)
```

```
selection_time = time.perf_counter() - start

def short_display(arr):
    return f"{arr[:10]} ........ {arr[-10:]}"

print("Bubble Sort Results:")
print(f"  Time Taken: {bubble_time:.6f} seconds")
print(f"  Comparisons: {bubble_comps}")
print("  Sorted Numbers:", short_display(bubble_sorted), "\n")

print("Selection Sort Results:")
print(f"  Time Taken: {selection_time:.6f} seconds")
print(f"  Comparisons: {selection_comps}")
print("  Sorted Numbers:", short_display(selection_sorted))
```

Result:

```
Bubble Sort Results:
  Time Taken: 0.078465 seconds
  Comparisons: 124750
  Sorted Numbers: [30, 33, 41, 49, 54, 61, 67, 84, 90, 101] ........ [9910, 9933, 9934, 9936, 9939, 9958, 9974, 9978, 9987,
9999]

Selection Sort Results:
  Time Taken: 0.042525 seconds
  Comparisons: 124750
  Sorted Numbers: [30, 33, 41, 49, 54, 61, 67, 84, 90, 101] ........ [9910, 9933, 9934, 9936, 9939, 9958, 9974, 9978, 9987,
9999]
```

SEARCHING

Code:

```
import random
import time

def short_display(arr):
    return f"{arr[:10]} ... {arr[-10:]}"

numbers = random.sample(range(1, 10000), 500)

def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

def binary_search(arr, target):
    numbers.sort()
    print("Sorted Numbers:", short_display(numbers), "\n")
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
```

```
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

target = random.choice(numbers)
print(f"Target to Search: {target}\n")

start_time = time.time()
linear_index = linear_search(numbers, target)
linear_time = time.time() - start_time
print(f"Linear Search -> Index: {linear_index}, Time: {linear_time:.8f} seconds")

start_time = time.time()
binary_index = binary_search(numbers, target)
binary_time = time.time() - start_time
print(f"Binary Search -> Index: {binary_index}, Time: {binary_time:.8f} seconds")

# ---------- Compare ----------
if linear_time > binary_time:
    print("\n✅ Binary Search is faster.")
elif binary_time > linear_time:
    print("\n✅ Linear Search is faster (unusual for large sorted data).")
else:
    print("\nBoth took the same time.")
```

Output:

```
Target to Search: 5534

Linear Search -> Index: 208, Time: 0.00054860 seconds
Sorted Numbers: [40, 42, 50, 59, 93, 124, 148, 168, 187, 203] ... [9807, 9817, 9824, 9887, 9907, 9918, 9920, 9934, 9938, 999
8]

Binary Search -> Index: 300, Time: 0.00138402 seconds

✅ Linear Search is faster (unusual for large sorted data).
```